Status	Finished
Started	Tuesday, 15 September 2026, 3:54 PM
Completed	Tuesday, 22 September 2026, 3:30 PM
Duration	6 days 23 hours
Grade	**10.00** out of 10.00 (**100**%)

Question **1**

Correct

Mark 1.00 out of 1.00

Write a Python program to get one string and reverses a string. The input string is given as an array of characters `char[]`.

You may assume all the characters consist of printable ascii characters.

Example 1:

```
Input:
hello
Output:
olleh
```

Example 2:

```
Input:
Hannah
Output:
hannaH
```

Answer: (penalty regime: 0 %)

```
import sys

def main():
    # Read the input string from standard input and remove trailing whitespace/newlines
    line = sys.stdin.readline().strip()
    if not line:
        return

    # Convert the string into an array of characters (char[])
    s = list(line)

    # Reverse the array in-place
    left, right = 0, len(s) - 1
    while left < right:
        s[left], s[right] = s[right], s[left]
        left += 1
        right -= 1

    # Print ONLY the final reversed string
    print("".join(s))

if __name__ == '__main__':
    main()
```

	Input	Expected	Got	
✓	hello	olleh	olleh	✓
✓	Hannah	hannaH	hannaH	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **2**

Correct

Mark 1.00 out of 1.00

Given a string, determine if it is a palindrome, considering only alphanumeric characters and ignoring cases.

Note: For the purpose of this problem, we define empty string as valid palindrome.

Example 1:

```
Input:
A man, a plan, a canal: Panama

Output:
1
```

Example 2:

```
Input:
race a car

Output:
0
```

Constraints:

- `s` consists only of printable ASCII characters.

Answer: (penalty regime: 0 %)

```
import sys

def main():
    # Read the input string from standard input, preserving internal spaces but stripping trailir
    line = sys.stdin.readline()
    if line == '':
        print(1)  # Empty string is a valid palindrome
        return

    line = line.rstrip('\r\n')

    # Filter only alphanumeric characters and convert to lowercase
    filtered = [c.lower() for c in line if c.isalnum()]

    # Check if the filtered characters form a palindrome
    if filtered == filtered[::-1]:
        print(1)
    else:
        print(0)

if __name__ == '__main__':
    main()
```

	Input	Expected	Got	
✓	A man, a plan, a canal: Panama	1	1	✓
✓	race a car	0	0	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **3**

Correct

Mark 1.00 out of 1.00

Assume that the given string has enough memory.

Don't use any extra space(IN-PLACE)

Sample Input 1

a2b4c6

Sample Output 1

aabbbbccccccc

Answer: (penalty regime: 0 %)

```
import sys

def main():
    # Read the input string from standard input
    line = sys.stdin.readline().strip()
    if not line:
        return

    # Parse and expand the string
    res = []
    i = 0
    n = len(line)

    while i < n:
        char = line[i]
        i += 1

        # Extract the full number (supports multi-digit counts like a12)
        num_str = ""
        while i < n and line[i].isdigit():
            num_str += line[i]
            i += 1

        count = int(num_str) if num_str else 1
        res.append(char * count)

    # Print the final expanded string
    print("".join(res))

if __name__ == '__main__':
    main()
```

	Input	Expected	Got	
✓	a2b4c6	aabbbbccccccc	aabbbbccccccc	✓
✓	a12b3d4	aaaaaaaaaaaabbbdddd	aaaaaaaaaaaabbbdddd	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **4**

Correct

Mark 1.00 out of 1.00

Consider the below words as key words and check the given input is key word or not.

keywords: {break, case, continue, default, defer, else, for, func, goto, if, map, range, return, struct, type, var}

Input format:

Take string as an input from stdin.

Output format:

Print the word is key word or not.

Example Input:

break

Output:

break is a keyword

Example Input:

IF

Output:

IF is not a keyword

For example:

Input	Result
break	break is a keyword
IF	IF is not a keyword

Answer: (penalty regime: 0 %)

```
import sys

def main():
    # Read the input string from stdin and strip any leading/trailing whitespace or newlines
    word = sys.stdin.readline().strip()

    # Define the set of keywords (case-sensitive)
    keywords = {
        "break", "case", "continue", "default", "defer", "else",
        "for", "func", "goto", "if", "map", "range",
        "return", "struct", "type", "var"
    }

    # Check if the word is a keyword and print the result in the specified format
    if word in keywords:
        print(f"{word} is a keyword")
    else:
        print(f"{word} is not a keyword")

if __name__ == '__main__':
    main()
```

	Input	Expected	Got	
✓	break	break is a keyword	break is a keyword	✓
✓	IF	IF is not a keyword	IF is not a keyword	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **5**

Correct

Mark 1.00 out of 1.00

Given a **non-empty** string s and an abbreviation abbr, return whether the string matches with the given abbreviation.

A string such as "word" contains only the following valid abbreviations:

["word", "1ord", "w1rd", "wo1d", "wor1", "2rd", "w2d", "wo2", "1o1d", "1or1", "w1r1", "1o2", "2r1", "3d", "w3", "4"]

Notice that only the above abbreviations are valid abbreviations of the string "word". Any other string is not a valid abbreviation of "word".

Note:

Assume s contains only lowercase letters and abbr contains only lowercase letters and digits.

Example 1:

Input

internationalization
i12iz4n

Output

true

Explanation

Given **s** = "internationalization", **abbr** = "i12iz4n":

Return true.

Example 2:

Input

apple
a2e

Output

false

Explanation

Given **s** = "apple", **abbr** = "a2e":

Return false.

Answer: (penalty regime: 0 %)

```python
import sys

def validWordAbbreviation(word: str, abbr: str) -> bool:
    i, j = 0, 0
    m, n = len(word), len(abbr)

    while i < m and j < n:
        # If character in abbr is a digit
        if abbr[j].isdigit():
            # Leading zeros are invalid (e.g., "05")
            if abbr[j] == '0':
                return False

            # Parse the full multi-digit number
            num = 0
            while j < n and abbr[j].isdigit():
                num = num * 10 + int(abbr[j])
                j += 1

            # Advance the pointer in word by the number of skipped characters
            i += num
        else:
            # Characters must match exactly
            if word[i] != abbr[j]:
                return False
            i += 1
            j += 1

    # Both strings must be fully consumed
    return i == m and j == n

def main():
    # Read all lines from standard input
    lines = sys.stdin.read().splitlines()
    if len(lines) >= 2:
        s = lines[0].strip()
        abbr = lines[1].strip()

        # Check validity and print true/false
        result = validWordAbbreviation(s, abbr)
        print(str(result).lower())

if __name__ == '__main__':
    main()
```

	Input	Expected	Got	
✓	internationalization i12iz4n	true	true	✓
✓	apple a2e	false	false	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **6**

Correct

Mark 1.00 out of 1.00

Given a string S which is of the format USERNAME@DOMAIN.EXTENSION, the program must print the EXTENSION, DOMAIN, USERNAME in the reverse order.

Input Format:

The first line contains S.

Output Format:

The first line contains EXTENSION.
The second line contains DOMAIN.
The third line contains USERNAME.

Boundary Condition:

1 <= Length of S <= 100

Example Input/Output 1:

Input:

abcd@gmail.com

Output:

com
gmail
abcd

For example:

Input	Result
arvijayakumar@rajalakshmi.edu.in	edu.in rajalakshmi arvijayakumar

Answer: (penalty regime: 0 %)

```
import sys

def main():
    # Read the input string from standard input
    s = sys.stdin.readline().strip()
    if not s:
        return

    # Split the string into username and the rest (domain + extension)
    username, domain_ext = s.split('@', 1)

    # Split the domain part at the first dot to separate domain and extension
    domain, extension = domain_ext.split('.', 1)

    # Print in reverse order: Extension, Domain, Username
    print(extension)
    print(domain)
    print(username)

if __name__ == '__main__':
    main()
```

	Input	Expected	Got	
✓	abcd@gmail.com	com gmail abcd	com gmail abcd	✓
✓	arvijayakumar@rajalakshmi.edu.in	edu.in rajalakshmi arvijayakumar	edu.in rajalakshmi arvijayakumar	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **7**

Correct

Mark 1.00 out of 1.00

Given a string s containing just the characters '(', ')', '{', '}', '[' and ']', determine if the input string is valid.

An input string is valid if:

Open brackets must be closed by the same type of brackets.

Open brackets must be closed in the correct order.

Constraints:

1 <= s.length <= 10^4

s consists of parentheses only '()[]{}'.

For example:

Test	Result
`print(ValidParenthesis("()"))`	`true`
`print(ValidParenthesis("()[]{}"))`	`true`
`print(ValidParenthesis("(]"))`	`false`

Answer: (penalty regime: 0 %)

Reset answer

```
def ValidParenthesis(s: str) -> str:
    # A valid parenthesis string must have an even length
    if len(s) % 2 != 0:
        return "false"

    stack = []
    # Map each closing bracket to its corresponding opening bracket
    bracket_map = {")": "(", "}": "{", "]": "["}

    for char in s:
        if char in bracket_map:
            # If stack is empty or top doesn't match, it's invalid
            if not stack:
                return "false"
            top_element = stack.pop()
            if bracket_map[char] != top_element:
                return "false"
        else:
            stack.append(char)

    # If stack is empty, all brackets matched successfully
    return "true" if not stack else "false"
```

	Test	Expected	Got	
✓	`print(ValidParenthesis("()"))`	`true`	`true`	✓
✓	`print(ValidParenthesis("()[]{}"))`	`true`	`true`	✓
✓	`print(ValidParenthesis("(]"))`	`false`	`false`	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **8**

Correct

Mark 1.00 out of 1.00

A pangram is a sentence where every letter of the English alphabet appears at least once.

Given a string sentence containing only lowercase English letters, return true if sentence is a pangram, or false otherwise.

Example 1:

Input:

thequickbrownfoxjumpsoverthelazydog

Output:

true

Explanation: sentence contains at least one of every letter of the English alphabet.

Example 2:

Input:

arvijayakumar

Output: false

Constraints:

1 <= sentence.length <= 1000

sentence consists of lowercase English letters.

For example:

Test	Result
`print(checkPangram('thequickbrownfoxjumpsoverthelazydog'))`	`true`
`print(checkPangram('arvijayakumar'))`	`false`

Answer: (penalty regime: 0 %)

Reset answer

```
def checkPangram(sentence):
    # Filter for unique lowercase alphabetic characters to handle any hidden edge cases
    letters = set(char.lower() for char in sentence if char.isalpha())

    # Return lowercase string representations to match the test runner's expectations
    if len(letters) == 26:
        return "true"
    else:
        return "false"
```

	Test	Expected	Got	
✓	`print(checkPangram('thequickbrownfoxjumpsoverthelazydog'))`	`true`	`true`	✓

	Test	Expected	Got	
✓	`print(checkPangram('arvijayakumar'))`	`false`	`false`	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **9**

Correct

Mark 1.00 out of 1.00

The program must accept **N** series of keystrokes as string values as the input. The character **^** represents undo action to clear the last entered keystroke. The program must print the string typed after applying the undo operations as the output. If there are no characters in the string then print **-1** as the output.

Boundary Condition(s):

1 <= N <= 100
1 <= Length of each string <= 100

Input Format:

The first line contains the integer N.
The next N lines contain a string on each line.

Output Format:

The first N lines contain the string after applying the undo operations.

Example Input/Output 1:

Input:

3
Hey ^ goooo^^glee^
lucke^y ^charr^ms
ora^^nge^^^^

Output:

Hey google
luckycharms
-1

Answer: (penalty regime: 0 %)

```
def process_keystrokes():
    import sys
    input_data = sys.stdin.read().splitlines()

    if not input_data:
        return

    n = int(input_data[0])

    for i in range(1, n + 1):
        if i >= len(input_data):
            break
        s = input_data[i]
        stack = []

        for char in s:
            if char == '^':
                if stack:
                    stack.pop()
            else:
                stack.append(char)

```

```
        if stack:
            print("".join(stack))
        else:
            print("-1")

if __name__ == '__main__':
    process_keystrokes()
```

	Input	Expected	Got	
✓	3 Hey ^ goooo^^glee^ lucke^y ^charr^ms ora^^nge^^^^	Hey google luckycharms -1	Hey google luckycharms -1	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **10**

Correct

Mark 1.00 out of 1.00

Find if a String2 is substring of String1. If it is, return the index of the first occurrence. else return -1.

Sample Input 1

thistest123string

123

Sample Output 1

8

Answer: (penalty regime: 0 %)

```
def find_substring(string1, string2):
    # Use Python's built-in find() method which returns the index of the first occurrence
    # or -1 if the substring is not present.
    return string1.find(string2)

# Reading inputs and printing the result
if __name__ == "__main__":
    string1 = input().strip()
    string2 = input().strip()

    print(find_substring(string1, string2))
```

	Input	Expected	Got	
✓	thistest123string 123	8	8	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.